NYC Taxi AI Analytics Project

Deliverable 2 Full Submission Report: Model Optimization, Metrics, Ethics, Outputs, and Code

Field	Value
Repository branch	project_day_2
Project objective	Predict whether a cleaned NYC yellow taxi trip lasts at least 30 minutes
Final model	Optimized histogram gradient boosting classifier
Final test accuracy	94.17%
Final test precision	62.01%
Team members	Soumith Kumar Ananthula, Varun Reddy Beesam, Vivek Doppalapudi

This PDF is designed as a one-file submission packet. It includes the written deliverable, dataset and preprocessing summary, hyperparameter tuning details, final evaluation metrics, figures, run instructions, dependencies, generated output inventory, and core source code.

Submission Contents

1. Executive Summary

The Day 2 work refined the Day 1 taxi long-trip classifier by adding a validation split, testing histogram gradient boosting candidates, tuning model hyperparameters, and selecting a recall-protected probability threshold. The final model predicts whether a trip is likely to last 30 minutes or longer.

Compared with the Day 1 logistic regression model, the optimized model increased accuracy from 0.9176 to 0.9417, precision from 0.5183 to 0.6201, and recall from 0.8647 to 0.8685. F1 improved from 0.6482 to 0.7236, and ROC-AUC improved from 0.9622 to 0.9751. The final threshold was lowered from the earlier strict version so long-trip recall stays above the logistic reference.

- Primary gain: fewer false long-trip alerts while preserving long-trip recall.
- False positives dropped from 1,693 to 1,121.
- False negatives dropped from 285 to 277.
- Recommended use: aggregate planning and decision support, not automated ride denial or neighborhood-level service restriction.

2. Deliverable 2 Written Report

Deliverable 2: Model Optimization, Real-World Impact, and Ethical Evaluation

Project Title

Predicting Long NYC Yellow Taxi Trips for Dispatch and Congestion-Aware Planning

Objective

This second deliverable refines the Day 1 NYC taxi long-trip model, tunes hyperparameters, evaluates the final optimized model, and reflects on real-world value, ethical limits, and future improvements. The project still predicts the binary target long_trip, where 1 means a cleaned yellow taxi trip lasted at least 30 minutes and 0 means it lasted under 30 minutes.

The dataset remains the official NYC Taxi and Limousine Commission Yellow Taxi Trip Record file for January 2024. The pipeline cleaned 2,964,624 raw rows down to 2,684,599 realistic trips, then used a deterministic 120,000-row modeling sample with random_state=42. Long trips represented 8.78% of the modeling sample, so model quality must be judged with precision, recall, F1, F0.5, and ROC-AUC in addition to accuracy.

1. Hyperparameter Tuning

The Day 1 model used a class-balanced logistic regression classifier. That model was useful because it was interpretable and had strong long-trip recall, but its precision was modest. In this use case, low precision means the system would warn dispatch teams about too many trips that do not actually become long trips. For Deliverable 2, the optimization goal was to improve accuracy and precision while still retaining enough recall for planning.

The final tuning workflow used a 60/20/20 stratified train-validation-test split. The training set was used to fit candidate models, the validation set was used to choose hyperparameters and the classification threshold, and the test set was held out for final evaluation. This is more rigorous than tuning directly on the test set because it keeps the final test metrics independent from the model-selection step.

The optimized model family was scikit-learn's histogram gradient boosting classifier. This model was selected because it can capture nonlinear relationships between trip distance, pickup time, taxi zones, airport trips, and

long-trip likelihood. Categorical fields were ordinal-encoded inside a scikit-learn Pipeline, and the classifier was told which transformed columns were categorical. This avoids treating taxi zone IDs as simple continuous numeric values.

Five candidate configurations were evaluated by changing max_iter, learning_rate, max_leaf_nodes, l2_regularization, and class_weight. For each candidate, the validation probabilities were searched across thresholds from 0.05 to 0.95. After review, the threshold selection metric was changed to F0.5 with validation recall at or above 0.865. F0.5 still weights precision more heavily than recall, but the higher recall floor keeps long-trip recall above the Day 1 logistic regression reference.

The selected candidate was:

- Model: optimized histogram gradient boosting
- Candidate: `hgb_depth31_weight3`
- `max_iter`: 400
- `learning_rate`: 0.03
- `max_leaf_nodes`: 31
- `l2_regularization`: 0.01
- `class_weight`: `{0: 1, 1: 3}`
- Decision threshold: `0.35`

The validation tuning table is saved in reports/tuning_results.csv. The selected model had the strongest validation F0.5 among candidates that met the recall floor.

2. Final Model Evaluation

The held-out test set contained 24,000 trips with the same 8.78% long-trip rate as the full modeling sample. A most-frequent baseline still appears accurate because most trips are short, but it fails the real business objective: it never predicts a long trip. The baseline reached 91.22% accuracy with 0.00 precision, 0.00 recall, and 0.00 F1 for the long-trip class.

The Day 1 logistic regression reference model had strong recall but too many false positives:

The optimized model is a stronger final model for the revised Day 2 objective because it improves precision while also protecting recall. Accuracy improved from 0.9176 to 0.9417, precision improved from 0.5183 to 0.6201, recall improved slightly from 0.8647 to 0.8685, F1 improved from 0.6482 to 0.7236, and ROC-AUC improved from 0.9622 to 0.9751 compared with the logistic regression reference. This means the model now catches slightly more actual long trips and makes fewer false long-trip warnings than the Day 1 model.

The final confusion matrix on the test set was:

Compared with the logistic model, the optimized model reduced false positives from 1,693 to 1,121 and reduced false negatives from 285 to 277. This is a better operational balance than the earlier precision-only threshold because the final model keeps long-trip recall slightly above logistic regression. It still produces more false positives than a very strict threshold, but that is an acceptable tradeoff for a planning use case where missed long trips can weaken staffing and service decisions.

Feature importance was estimated with held-out permutation importance using ROC-AUC as the scoring metric. The most important feature was trip_distance, which is expected because longer distances are more likely to last 30 minutes or more. Time and location features also mattered: pickup_hour, DOLocationID, PULocationID, and pickup_dayofweek were the next strongest contributors. This confirms that the final model is learning plausible transportation patterns rather than relying on a single artifact.

Generated evaluation artifacts are saved in:

- `reports/model_metrics.csv`
- `reports/metrics.json`
- `reports/tuning_results.csv`
- `reports/model_feature_importance.csv`
- `reports/figures/confusion_matrix.png`
- `reports/figures/feature_importance.png`

3. Ethical Considerations

This model does not use names, phone numbers, exact addresses, race, gender, or other direct personal identifiers. However, it still uses pickup and dropoff taxi zone IDs. Location data can reflect neighborhood-level patterns related to income, commuting access, airport travel, tourism, disability access, and historical service availability. Because of that, the model must be treated as a decision-support tool rather than an automated authority.

The main ethical risk is misuse. A long-trip prediction could be helpful for dispatch planning, but it could become harmful if used to discourage drivers from accepting certain trips, avoid certain neighborhoods, penalize drivers for route outcomes, or justify unequal service. The model also learns from historical taxi behavior, so it may reproduce existing patterns instead of describing what fair transportation service should look like.

The final model's higher precision and protected recall reduce both unnecessary alerts and missed long-trip cases compared with the Day 1 reference, but ethical evaluation cannot stop at aggregate accuracy. Future audits should measure false-positive and false-negative rates by pickup zone, dropoff zone, time of day, weekday/weekend, and airport-trip status. If errors concentrate in specific neighborhoods or time periods, the model should be revised before being used in operations. Clear documentation should also state that the probability is an estimate, not a guarantee.

4. Real-World Application

A practical use of this model is an operational dashboard for taxi dispatch teams, fleet managers, airport queue monitors, or city transportation analysts. Before or near the beginning of a trip, the system can estimate whether the trip is likely to last at least 30 minutes using distance, pickup hour, day of week, airport flag, vendor, rate code, and pickup/dropoff zones. The output can be shown as a probability and a long-trip flag based on the tuned threshold.

For dispatch teams, the model can help estimate when vehicles may be unavailable for longer periods. This can improve driver shift planning, airport staging, and service coverage in high-demand areas. For city analysts, aggregated long-trip predictions can help identify times and zones where congestion or long-distance travel patterns may be increasing. For customer-facing systems, the probability could support better wait-time messaging, as long as it is framed carefully and not treated as a precise duration estimate.

The model should be deployed with human oversight. A dashboard should show aggregate counts, probability bands, and uncertainty rather than only a hard yes/no label. For example, trips could be grouped as low, medium, or high long-trip risk. Dispatchers could use those signals to plan coverage, but the system should not automatically deny service, change driver pay, or rank neighborhoods. The tuned threshold of 0.35 is appropriate for the revised objective because it improves precision while keeping recall at least as strong as the Day 1 logistic model. The threshold should still be revisited if business priorities change.

The current pipeline is reproducible and collaboration-ready. It uses pandas and NumPy for data manipulation, Matplotlib and Seaborn for visualization, scikit-learn for modeling and evaluation, Google Colab-compatible notebooks for collaboration, and GitHub for version control. Dask remains optional for future scaling if the team expands from a 120,000-row modeling sample to multiple months or full-year training.

Operational deployment would require several additional steps. First, the model should be validated on months beyond January 2024 to check seasonality and drift. Second, probability calibration should be tested so probability

values can be trusted for dashboard bands. Third, subgroup error monitoring should be added by zone and time period. Fourth, model retraining should be scheduled when travel patterns change due to weather, holidays, construction, major events, or policy changes.

5. Final Thoughts and Conclusion

Deliverable 2 improved the model from an interpretable recall-focused baseline into a stronger optimized classifier. The final histogram gradient boosting model achieved 94.17% accuracy, 62.01% precision, 86.85% recall, 72.36% F1, and 97.51% ROC-AUC on the held-out test set. The biggest practical improvement is that the model now reduces false long-trip alerts while still catching slightly more actual long trips than the Day 1 logistic regression model.

The project also shows why optimization is not only about raising one metric. The earlier strict threshold improved precision sharply but did not protect long-trip recall. The revised threshold is a better final choice because it keeps recall at the logistic model's level while still improving precision, accuracy, F1, and ROC-AUC. In a different operational setting, the threshold could still be adjusted to favor even higher recall or even higher precision.

The final recommendation is to use this model for aggregate planning, not automated service decisions. It can help teams anticipate long-trip pressure, understand transportation patterns, and plan resources more effectively. Future work should validate the model across multiple months, add contextual data such as weather and events, calibrate probabilities, and run fairness/error audits by zone and time. With those safeguards, the model can become a useful real-world analytics tool while respecting the ethical limits of transportation prediction.

3. Dataset and Preprocessing Evidence

The pipeline used the official NYC TLC January 2024 yellow taxi parquet file. It loaded 2,964,624 raw rows, retained 2,684,599 realistic rows after cleaning, and used a deterministic 120,000-row sample for modeling.

Preprocessing step	Rows removed	Rows after
Kept trips with pickup timestamps in January 2024.	18	2,964,606
Removed trips shorter than 1 minute or longer than 180 minutes.	37,104	2,927,502
Removed trips with zero, near-zero, negative, or extreme distances.	38,743	2,888,759
Kept trips with passenger_count from 1 to 6.	147,459	2,741,300
Removed trips with non-positive fare_amount.	29,647	2,711,653
Kept standard TLC rate codes 1 through 6.	27,054	2,684,599
Selected deterministic modeling sample with random_state=42.	2,564,599	120,000

Feature Engineering Summary

Feature	Type	Purpose
trip_duration_min	derived numeric	Used for cleaning and to create the long_trip target; not used as a model input.
pickup_hour	derived numeric	Captures daily travel and congestion cycles.
pickup_dayofweek	derived numeric	Captures weekday/weekend and commute-pattern differences.
is_weekend	derived binary	Flags weekend operating patterns.
airport_trip	derived binary	Flags trips involving major airport zones.
long_trip	target binary	Prediction target: 1 for long trips, 0 otherwise.
VendorID, RatecodeID, PULocationID, DOLocationID	encoded categorical	Prevents ID values from being treated as ordered numeric quantities.

4. Hyperparameter Tuning Details

The selected model was Optimized histogram gradient boosting. Five histogram-gradient-boosting candidates were tested on the validation set. The selected candidate was hgb_depth31_weight3 with max_iter = 400, learning_rate = 0.03, max_leaf_nodes = 31, l2_regularization = 0.01, class_weight = {0: 1, 1: 3}, and a decision threshold of 0.35.

The selection metric was validation F0.5 with recall >= 0.865. F0.5 gives more weight to precision than recall, which matches the assignment request to make the model more accurate and precise, while the higher recall floor keeps long-trip recall above the Day 1 logistic reference.

Candidate	Threshold	Max iter	Learning rate	Leaves	L2	Val precision	Val recall	Selected
hgb_depth31_weight4	0.39	250	0.06	31	0.0	0.6048	0.8657	False
hgb_depth63_weight4	0.37	300	0.05	63	0.01	0.6004	0.8671	False
hgb_depth31_balanced	0.60	300	0.05	31	0.1	0.5960	0.8662	False
hgb_depth63_weight6	0.44	200	0.08	63	0.1	0.5978	0.8657	False
hgb_depth31_weight3	0.35	400	0.03	31	0.01	0.6110	0.8662	True

5. Final Model Metrics and Changes

Model	Accuracy	Precision	Recall	F1	F0.5	ROC-AUC
Most frequent baseline	0.9122	0.0000	0.0000	0.0000	0.0000	0.5000
Balanced logistic regression	0.9176	0.5183	0.8647	0.6482	0.5635	0.9622
Optimized histogram gradient boosting	0.9417	0.6201	0.8685	0.7236	0.6578	0.9751

Metric	Day 1 logistic	Day 2 optimized	Change
Accuracy	0.9176	0.9417	+2.41 pts
Precision	0.5183	0.6201	+10.18 pts
Recall	0.8647	0.8685	+0.38 pts
F1	0.6482	0.7236	+7.54 pts
ROC-AUC	0.9622	0.9751	+1.29 pts
False positives	1,693	1,121	-572
False negatives	285	277	-8

Final classification report

Class	Precision	Recall	F1-score	Support
Under 30 min	0.9868	0.9488	0.9674	21,893
30+ min	0.6201	0.8685	0.7236	2,107
Weighted avg	0.9546	0.9417	0.9460	24,000

6. Figures and Model Interpretation

Figure 1. Cleaned trip duration distribution with the 30-minute threshold.

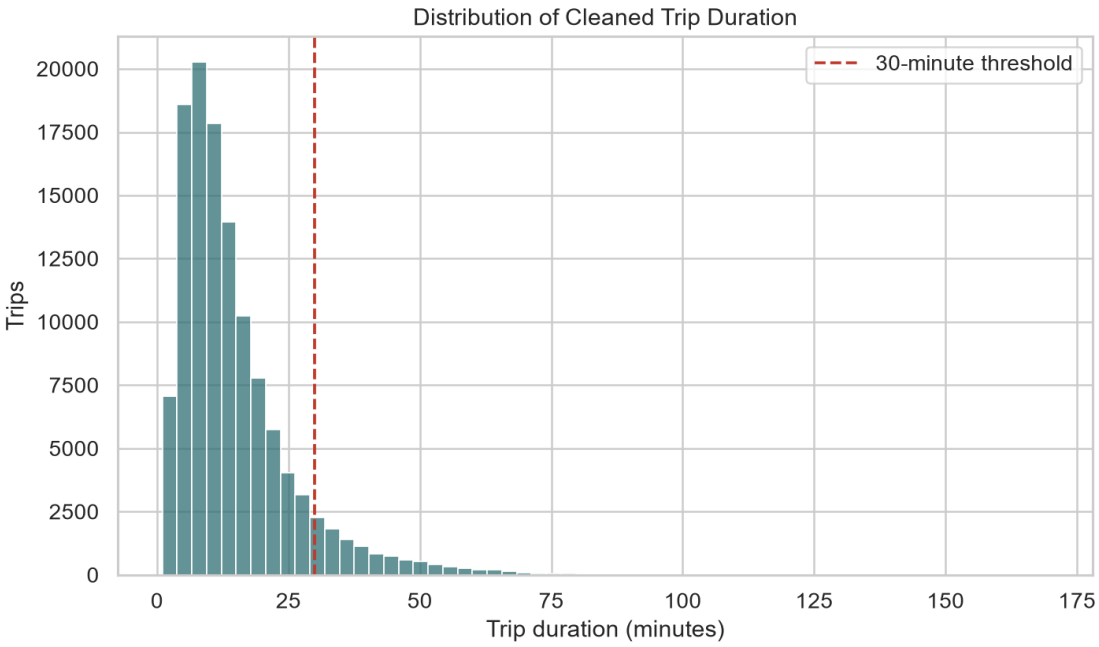


Figure 2. Long-trip rate by pickup hour.

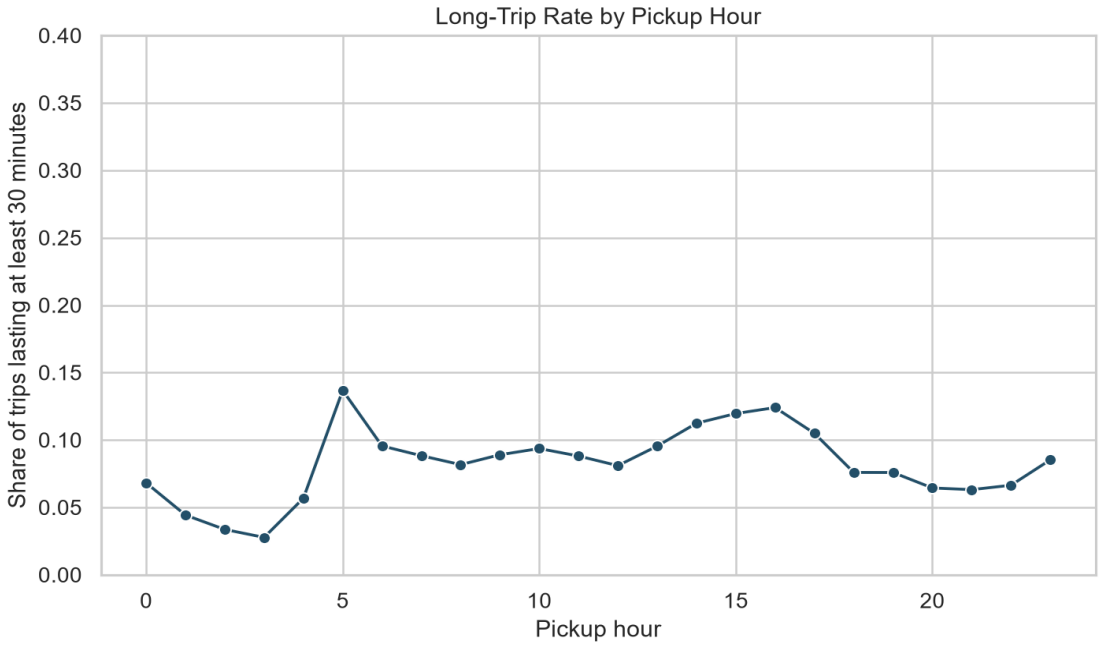


Figure 3. Optimized model confusion matrix on the held-out test set.

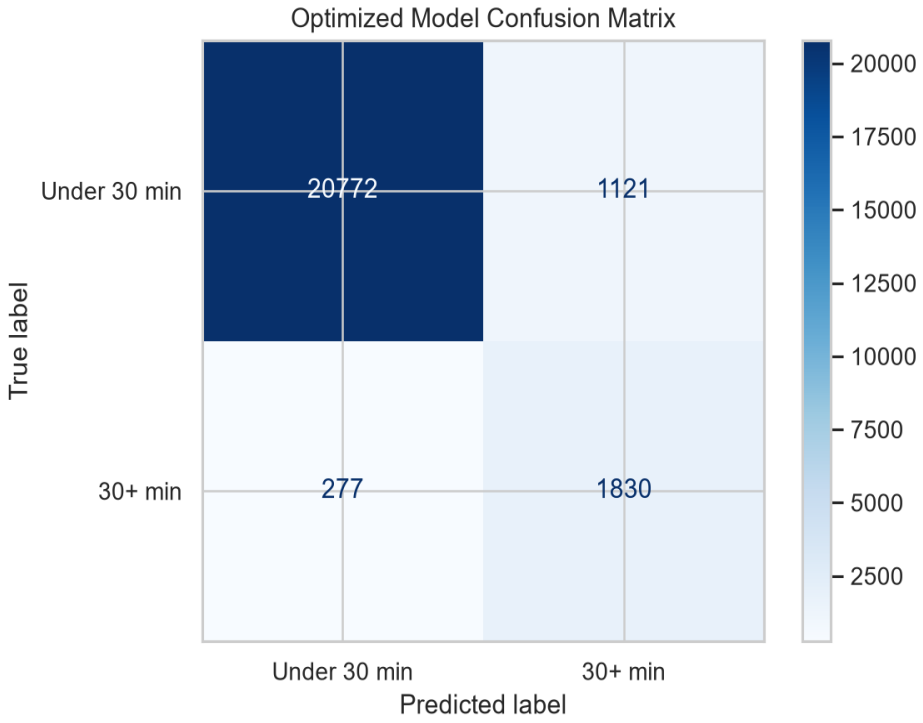
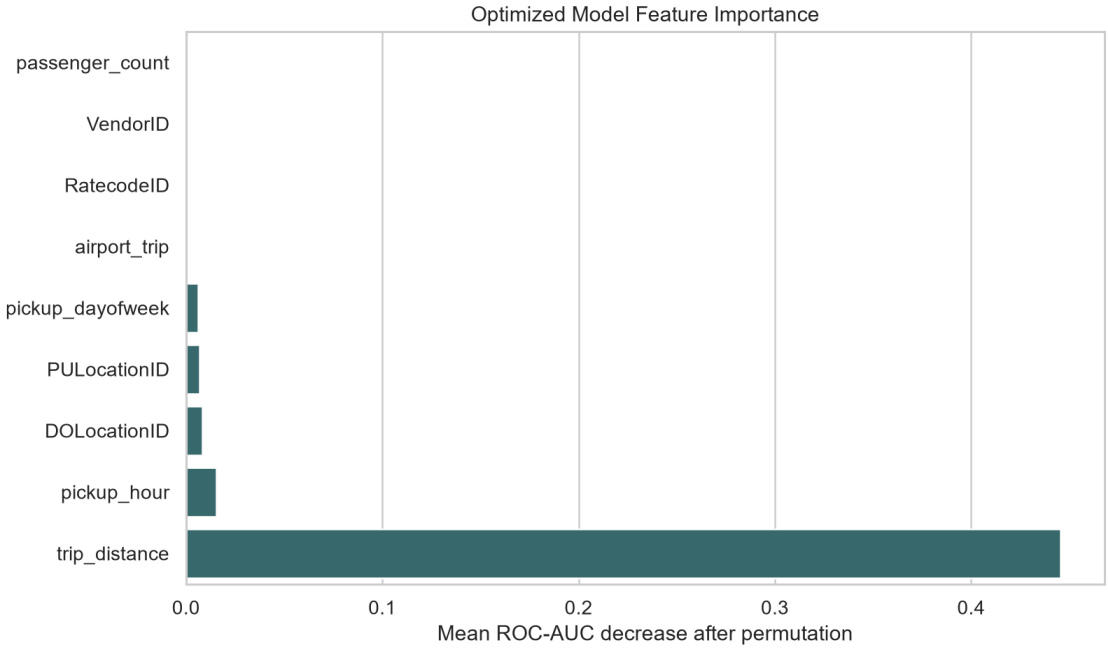


Figure 4. Permutation feature importance for the optimized model.



Feature importance values

Feature	Importance mean	Importance std	Scoring
trip_distance	0.445536	0.006913	roc_auc
pickup_hour	0.015539	0.000217	roc_auc
DOLocationID	0.008371	0.000813	roc_auc
PULocationID	0.006832	0.000909	roc_auc
pickup_dayofweek	0.006099	0.000236	roc_auc
airport_trip	0.000092	0.000013	roc_auc
RatecodeID	0.000021	0.000010	roc_auc
VendorID	0.000018	0.000012	roc_auc
passenger_count	0.000003	0.000004	roc_auc
is_weekend	0.000000	0.000000	roc_auc

7. Reproducibility Guide

Google Colab setup

```
!git clone --branch project_day_2 https://github.com/soumithkumara/project_day_1.git
%cd project_day_1
!pip install -r requirements.txt
!python src/nyc_taxi_long_trip_model.py
```

Recommended notebook

Open notebooks/nyc_taxi_long_trip_model_day2.ipynb for a fresh Colab run. The setup cell forces the project_day_2 branch and prints the active branch and commit.

Local run

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe src\nyc_taxi_long_trip_model.py
```

Generated artifact inventory

Path	Purpose
reports/assignment_report.md	Written Deliverable 2 report
reports/NYC_Taxi_Deliverable_2_Model_Optimization_Report.docx	Word report version
reports/metrics.json	Dataset summary, split summary, metrics, and classification report
reports/model_metrics.csv	Baseline, logistic, and optimized model metrics
reports/tuning_results.csv	Validation tuning candidates and selected model
reports/model_feature_importance.csv	Permutation importance for optimized model
reports/figures/	EDA and model evaluation figures
src/nyc_taxi_long_trip_model.py	Main training, tuning, evaluation, and artifact pipeline
notebooks/predict_new_taxi_trip.ipynb	Notebook for new-trip prediction

Appendix A: requirements.txt

```
pandas==3.0.3
numpy==2.4.6
matplotlib==3.11.0
seaborn==0.13.2
scikit-learn==1.9.0
pyarrow==24.0.0
joblib==1.5.3
reportlab==5.0.0
```

Appendix B: Complete Main Training Pipeline

This is the core project code that downloads/loads the data, preprocesses trips, tunes the model, evaluates metrics, writes artifacts, and saves the optimized model.

```
"""Build an initial NYC taxi long-trip risk model.

The pipeline downloads the official NYC TLC yellow taxi parquet file, cleans
trip records, creates features, trains an interpretable classifier, and writes
figures/metrics for the assignment report.
"""

from __future__ import annotations

import argparse
import json
import math
import urllib.request
from dataclasses import dataclass
from pathlib import Path
from typing import Any

import joblib
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.compose import ColumnTransformer
from sklearn.dummy import DummyClassifier
from sklearn.ensemble import HistGradientBoostingClassifier
from sklearn.impute import SimpleImputer
from sklearn.inspection import permutation_importance
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    ConfusionMatrixDisplay,
    accuracy_score,
    classification_report,
    confusion_matrix,
    fbeta_score,
    f1_score,
    precision_score,
    recall_score,
    roc_auc_score,
)
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder, StandardScaler


PROJECT_ROOT = Path(__file__).resolve().parents[1]
DEFAULT_DATA_URL = (
    "https://d37ci6vzurychx.cloudfront.net/trip-data/"
    "yellow_tripdata_2024-01.parquet"
)
DEFAULT_RAW_PATH = PROJECT_ROOT / "data" / "raw" / "yellow_tripdata_2024-01.parquet"
DEFAULT_REPORT_DIR = PROJECT_ROOT / "reports"
DEFAULT_MODEL_DIR = PROJECT_ROOT / "models"
RANDOM_STATE = 42

RAW_COLUMNS = [
    "VendorID",
    "tpep_pickup_datetime",
    "tpep_dropoff_datetime",
    "passenger_count",
    "trip_distance",
    "RatecodeID",
    "PULocationID",
    "DOLocationID",
    "payment_type",
    "fare_amount",
    "extra",
    "mta_tax",
    "tip_amount",
    "tolls_amount",
    "improvement_surcharge",
    "total_amount",
    "congestion_surcharge",
    "Airport_fee",
]

NUMERIC_FEATURES = [
    "trip_distance",
    "passenger_count",
    "pickup_hour",
    "pickup_dayofweek",
    "is_weekend",
    "airport_trip",
]

CATEGORICAL_FEATURES = [
    "VendorID",
```

```
    "RatecodeID",
    "PULocationID",
    "DOLocationID",
]

FEATURES = NUMERIC_FEATURES + CATEGORICAL_FEATURES
TARGET = "long_trip"
INITIAL_MODEL_NAME = "Balanced logistic regression"
FINAL_MODEL_NAME = "Optimized histogram gradient boosting"
DECISION_RECALL_FLOOR = 0.865
DECISION_BETA = 0.5

HGB_TUNING_CANDIDATES: list[dict[str, Any]] = [
    {
        "candidate": "hgb_depth31_weight4",
        "max_iter": 250,
        "learning_rate": 0.06,
        "max_leaf_nodes": 31,
        "l2_regularization": 0.0,
        "class_weight": {0: 1, 1: 4},
    },
    {
        "candidate": "hgb_depth63_weight4",
        "max_iter": 300,
        "learning_rate": 0.05,
        "max_leaf_nodes": 63,
        "l2_regularization": 0.01,
        "class_weight": {0: 1, 1: 4},
    },
    {
        "candidate": "hgb_depth31_balanced",
        "max_iter": 300,
        "learning_rate": 0.05,
        "max_leaf_nodes": 31,
        "l2_regularization": 0.1,
        "class_weight": "balanced",
    },
    {
        "candidate": "hgb_depth63_weight6",
        "max_iter": 200,
        "learning_rate": 0.08,
        "max_leaf_nodes": 63,
        "l2_regularization": 0.1,
        "class_weight": {0: 1, 1: 6},
    },
    {
        "candidate": "hgb_depth31_weight3",
        "max_iter": 400,
        "learning_rate": 0.03,
        "max_leaf_nodes": 31,
        "l2_regularization": 0.01,
        "class_weight": {0: 1, 1: 3},
    },
]

# TLC location IDs for Newark, JFK, and LaGuardia airport zones.
AIRPORT_ZONE_IDS = {1, 132, 138}


@dataclass(frozen=True)
class PipelineConfig:
    data_url: str
    raw_path: Path
    report_dir: Path
    model_dir: Path
    sample_size: int
    force_download: bool


def download_file(url: str, output_path: Path, force: bool = False) -> Path:
    """Download a file unless it already exists locally."""
    output_path.parent.mkdir(parents=True, exist_ok=True)
    if output_path.exists() and not force:
        return output_path

    def report_hook(block_num: int, block_size: int, total_size: int) -> None:
        if total_size <= 0:
            return
        downloaded = block_num * block_size
        percent = min(100, downloaded * 100 / total_size)
        if block_num % 100 == 0 or math.isclose(percent, 100):
            print(f"Downloading data: {percent:5.1f}%")

    urllib.request.urlretrieve(url, output_path, reporthook=report_hook)
    return output_path


def load_raw_data(path: Path) -> pd.DataFrame:
    """Read only the columns needed for this assignment."""
    return pd.read_parquet(path, columns=RAW_COLUMNS)


def _audit_filter(
```

```
    df: pd.DataFrame,
    mask: pd.Series,
    step: str,
    description: str,
    audit_rows: list[dict[str, Any]],
    raw_rows: int,
) -> pd.DataFrame:
    """Apply one preprocessing filter and record its row impact."""
    rows_before = len(df)
    filtered = df.loc[mask].copy()
    rows_after = len(filtered)
    audit_rows.append(
        {
            "step": step,
            "description": description,
            "rows_before": rows_before,
            "rows_after": rows_after,
            "rows_removed": rows_before - rows_after,
            "step_removed_pct": round(
                ((rows_before - rows_after) / rows_before) * 100, 4
            )
            if rows_before
            else 0.0,
            "retained_pct_of_raw": round((rows_after / raw_rows) * 100, 4)
            if raw_rows
            else 0.0,
        }
    )
    return filtered


def preprocess_with_audit(
    raw: pd.DataFrame,
) -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
    """Clean TLC records, create modeling features, and document each change."""
    df = raw.copy()
    raw_rows = len(df)
    audit_rows: list[dict[str, Any]] = [
        {
            "step": "01_raw_load",
            "description": "Loaded selected NYC TLC yellow taxi columns from parquet.",
            "rows_before": raw_rows,
            "rows_after": raw_rows,
            "rows_removed": 0,
            "step_removed_pct": 0.0,
            "retained_pct_of_raw": 100.0,
        }
    ]

    df["tpep_pickup_datetime"] = pd.to_datetime(df["tpep_pickup_datetime"])
    df["tpep_dropoff_datetime"] = pd.to_datetime(df["tpep_dropoff_datetime"])
    df["trip_duration_min"] = (
        df["tpep_dropoff_datetime"] - df["tpep_pickup_datetime"]
    ).dt.total_seconds() / 60
    audit_rows.append(
        {
            "step": "02_derive_duration",
            "description": "Converted pickup/dropoff timestamps and created trip_duration_min.",
            "rows_before": len(df),
            "rows_after": len(df),
            "rows_removed": 0,
            "step_removed_pct": 0.0,
            "retained_pct_of_raw": round((len(df) / raw_rows) * 100, 4),
        }
    )

    df = _audit_filter(
        df,
        (df["tpep_pickup_datetime"] >= "2024-01-01")
        & (df["tpep_pickup_datetime"] < "2024-02-01"),
        "03_filter_pickup_month",
        "Kept trips with pickup timestamps in January 2024.",
        audit_rows,
        raw_rows,
    )
    df = _audit_filter(
        df,
        df["trip_duration_min"].between(1, 180),
        "04_filter_duration",
        "Removed trips shorter than 1 minute or longer than 180 minutes.",
        audit_rows,
        raw_rows,
    )
    df = _audit_filter(
        df,
        df["trip_distance"].between(0.1, 60),
        "05_filter_distance",
        "Removed trips with zero, near-zero, negative, or extreme distances.",
        audit_rows,
        raw_rows,
    )
    df = _audit_filter(
        df,
```

```
    df["passenger_count"].between(1, 6),
    "06_filter_passengers",
    "Kept trips with passenger_count from 1 to 6.",
    audit_rows,
    raw_rows,
)
df = _audit_filter(
    df,
    df["fare_amount"] > 0,
    "07_filter_positive_fare",
    "Removed trips with non-positive fare_amount.",
    audit_rows,
    raw_rows,
)
df = _audit_filter(
    df,
    df["total_amount"] > 0,
    "08_filter_positive_total",
    "Removed trips with non-positive total_amount.",
    audit_rows,
    raw_rows,
)
df = _audit_filter(
    df,
    df["RatecodeID"].between(1, 6),
    "09_filter_standard_ratecode",
    "Kept standard TLC rate codes 1 through 6.",
    audit_rows,
    raw_rows,
)
df = _audit_filter(
    df,
    df["PULocationID"].between(1, 265),
    "10_filter_pickup_zone",
    "Kept valid TLC pickup location IDs from 1 to 265.",
    audit_rows,
    raw_rows,
)
df = _audit_filter(
    df,
    df["DOLocationID"].between(1, 265),
    "11_filter_dropoff_zone",
    "Kept valid TLC dropoff location IDs from 1 to 265.",
    audit_rows,
    raw_rows,
)

df["pickup_hour"] = df["tpep_pickup_datetime"].dt.hour
df["pickup_dayofweek"] = df["tpep_pickup_datetime"].dt.dayofweek
df["is_weekend"] = df["pickup_dayofweek"].isin([5, 6]).astype(int)
df["airport_trip"] = (
    df["PULocationID"].isin(AIRPORT_ZONE_IDS)
    | df["DOLocationID"].isin(AIRPORT_ZONE_IDS)
).astype(int)
df[TARGET] = (df["trip_duration_min"] >= 30).astype(int)

for column in CATEGORICAL_FEATURES:
    df[column] = df[column].astype("Int64").astype(str)

audit_rows.append(
    {
        "step": "12_feature_engineering",
        "description": "Created time, weekend, airport, target, and categorical model
            fields.",
        "rows_before": len(df),
        "rows_after": len(df),
        "rows_removed": 0,
        "step_removed_pct": 0.0,
        "retained_pct_of_raw": round((len(df) / raw_rows) * 100, 4),
    }
)

feature_rows = [
    {
        "column": "trip_duration_min",
        "type": "derived numeric",
        "source": "tpep_dropoff_datetime - tpep_pickup_datetime",
        "purpose": "Used for cleaning and to create the long_trip target; not used as a model
            input.",
    },
    {
        "column": "pickup_hour",
        "type": "derived numeric",
        "source": "hour from tpep_pickup_datetime",
        "purpose": "Captures daily travel and congestion cycles.",
    },
    {
        "column": "pickup_dayofweek",
        "type": "derived numeric",
        "source": "day of week from tpep_pickup_datetime, Monday=0",
        "purpose": "Captures weekday/weekend and commute-pattern differences.",
    },
    {
```

```
            "column": "is_weekend",
            "type": "derived binary",
            "source": "pickup_dayofweek in Saturday or Sunday",
            "purpose": "Flags weekend operating patterns.",
        },
        {
            "column": "airport_trip",
            "type": "derived binary",
            "source": "pickup or dropoff zone in Newark, JFK, or LaGuardia TLC IDs",
            "purpose": "Flags trips involving major airport zones.",
        },
        {
            "column": TARGET,
            "type": "target binary",
            "source": "trip_duration_min >= 30",
            "purpose": "Prediction target: 1 for long trips, 0 otherwise.",
        },
        {
            "column": ", ".join(CATEGORICAL_FEATURES),
            "type": "converted categorical",
            "source": "integer IDs converted to strings",
            "purpose": "Prevents ID values from being treated as ordered numeric quantities.",
        },
    ]

    return df, pd.DataFrame(audit_rows), pd.DataFrame(feature_rows)


def clean_and_engineer_features(raw: pd.DataFrame) -> pd.DataFrame:
    """Clean TLC records and create modeling features."""
    df, _, _ = preprocess_with_audit(raw)
    return df


def make_logistic_model() -> Pipeline:
    """Create the initial interpretable logistic classification pipeline."""
    numeric_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler()),
        ]
    )
    categorical_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="most_frequent")),
            (
                "onehot",
                OneHotEncoder(handle_unknown="ignore", min_frequency=50),
            ),
        ]
    )
    preprocessor = ColumnTransformer(
        transformers=[
            ("num", numeric_transformer, NUMERIC_FEATURES),
            ("cat", categorical_transformer, CATEGORICAL_FEATURES),
        ]
    )
    classifier = LogisticRegression(
        class_weight="balanced",
        max_iter=1000,
        random_state=RANDOM_STATE,
    )
    return Pipeline(steps=[("preprocess", preprocessor), ("model", classifier)])


def make_hist_gradient_boosting_model(params: dict[str, Any]) -> Pipeline:
    """Create the optimized model family used during hyperparameter tuning."""
    numeric_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="median")),
        ]
    )
    categorical_transformer = Pipeline(
        steps=[
            ("imputer", SimpleImputer(strategy="most_frequent")),
            (
                "ordinal",
                OrdinalEncoder(handle_unknown="use_encoded_value", unknown_value=-1),
            ),
        ]
    )
    preprocessor = ColumnTransformer(
        transformers=[
            ("num", numeric_transformer, NUMERIC_FEATURES),
            ("cat", categorical_transformer, CATEGORICAL_FEATURES),
        ],
        verbose_feature_names_out=False,
    )
    categorical_indices = list(range(len(NUMERIC_FEATURES), len(FEATURES)))
    model_params = {key: value for key, value in params.items() if key != "candidate"}
    classifier = HistGradientBoostingClassifier(
        categorical_features=categorical_indices,
        random_state=RANDOM_STATE,
```

```
        **model_params,
    )
    return Pipeline(steps=[("preprocess", preprocessor), ("model", classifier)])


def make_model() -> Pipeline:
    """Create the default optimized classification pipeline."""
    return make_hist_gradient_boosting_model(HGB_TUNING_CANDIDATES[1])


def evaluate_predictions(
    y_true: pd.Series,
    y_pred: np.ndarray,
    y_score: np.ndarray,
    threshold: float | None = None,
) -> dict[str, Any]:
    """Calculate core classification metrics."""
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred, labels=[0, 1]).ravel()
    metrics = {
        "accuracy": round(float(accuracy_score(y_true, y_pred)), 4),
        "precision": round(float(precision_score(y_true, y_pred, zero_division=0)), 4),
        "recall": round(float(recall_score(y_true, y_pred, zero_division=0)), 4),
        "f1": round(float(f1_score(y_true, y_pred, zero_division=0)), 4),
        "f0_5": round(
            float(fbeta_score(y_true, y_pred, beta=0.5, zero_division=0)),
            4,
        ),
        "roc_auc": round(float(roc_auc_score(y_true, y_score)), 4),
        "true_negative": int(tn),
        "false_positive": int(fp),
        "false_negative": int(fn),
        "true_positive": int(tp),
    }
    if threshold is not None:
        metrics["decision_threshold"] = round(float(threshold), 4)
    return metrics


def threshold_search(
    y_true: pd.Series,
    y_score: np.ndarray,
    recall_floor: float = DECISION_RECALL_FLOOR,
    beta: float = DECISION_BETA,
) -> tuple[float, pd.DataFrame]:
    """Choose a probability threshold on validation data."""
    rows: list[dict[str, Any]] = []
    for threshold in np.round(np.arange(0.05, 0.951, 0.01), 2):
        y_pred = (y_score >= threshold).astype(int)
        precision = precision_score(y_true, y_pred, zero_division=0)
        recall = recall_score(y_true, y_pred, zero_division=0)
        rows.append(
            {
                "threshold": float(threshold),
                "accuracy": float(accuracy_score(y_true, y_pred)),
                "precision": float(precision),
                "recall": float(recall),
                "f1": float(f1_score(y_true, y_pred, zero_division=0)),
                "f0_5": float(
                    fbeta_score(y_true, y_pred, beta=beta, zero_division=0)
                ),
                "predicted_positive_rate": float(np.mean(y_pred)),
            }
        )

    threshold_frame = pd.DataFrame(rows)
    feasible = threshold_frame[threshold_frame["recall"] >= recall_floor]
    selection_frame = feasible if not feasible.empty else threshold_frame
    selected = selection_frame.sort_values(
        ["f0_5", "accuracy", "precision"],
        ascending=[False, False, False],
    ).iloc[0]
    return float(selected["threshold"]), threshold_frame


def tune_hist_gradient_boosting(
    x_train: pd.DataFrame,
    y_train: pd.Series,
    x_validation: pd.DataFrame,
    y_validation: pd.Series,
) -> tuple[Pipeline, float, dict[str, Any], pd.DataFrame]:
    """Tune gradient boosting hyperparameters and decision threshold."""
    tuning_rows: list[dict[str, Any]] = []
    best_model: Pipeline | None = None
    best_threshold = 0.5
    best_candidate: dict[str, Any] | None = None
    best_score_key = (-np.inf, -np.inf, -np.inf)

    for candidate in HGB_TUNING_CANDIDATES:
        model = make_hist_gradient_boosting_model(candidate)
        model.fit(x_train, y_train)
        validation_score = model.predict_proba(x_validation)[:, 1]
        threshold, _ = threshold_search(y_validation, validation_score)
        validation_pred = (validation_score >= threshold).astype(int)
```

```
            metrics = evaluate_predictions(
                y_validation,
                validation_pred,
                validation_score,
                threshold=threshold,
            )
            row = {
                "candidate": candidate["candidate"],
                "model": FINAL_MODEL_NAME,
                "selected_threshold": round(float(threshold), 4),
                "max_iter": candidate["max_iter"],
                "learning_rate": candidate["learning_rate"],
                "max_leaf_nodes": candidate["max_leaf_nodes"],
                "l2_regularization": candidate["l2_regularization"],
                "class_weight": json.dumps(candidate["class_weight"]),
                "selection_metric": "validation_f0_5_with_recall_floor",
                "validation_recall_floor": DECISION_RECALL_FLOOR,
                **{f"validation_{key}": value for key, value in metrics.items()},
            }
            tuning_rows.append(row)
            score_key = (
                metrics["f0_5"],
                metrics["accuracy"],
                metrics["precision"],
            )
            if score_key > best_score_key:
                best_score_key = score_key
                best_model = model
                best_threshold = threshold
                best_candidate = candidate

    if best_model is None or best_candidate is None:
        raise RuntimeError("No tuned model candidate was selected.")

    tuning_frame = pd.DataFrame(tuning_rows)
    tuning_frame["selected"] = tuning_frame["candidate"] == best_candidate["candidate"]
    return best_model, best_threshold, best_candidate, tuning_frame


def coefficient_summary(model: Pipeline) -> pd.DataFrame:
    """Return sorted feature coefficients from the fitted logistic model."""
    preprocessor = model.named_steps["preprocess"]
    classifier = model.named_steps["model"]
    feature_names = preprocessor.get_feature_names_out()
    coefficients = classifier.coef_.ravel()
    frame = pd.DataFrame(
        {
            "feature": feature_names,
            "coefficient": coefficients,
            "absolute_coefficient": np.abs(coefficients),
        }
    )
    frame["feature"] = (
        frame["feature"]
        .str.replace("num__", "", regex=False)
        .str.replace("cat__", "", regex=False)
        .str.replace("onehot__", "", regex=False)
    )
    frame["direction"] = np.where(
        frame["coefficient"] >= 0,
        "Higher long-trip risk",
        "Lower long-trip risk",
    )
    return frame.sort_values("absolute_coefficient", ascending=False)


def feature_importance_summary(
    model: Pipeline,
    x_test: pd.DataFrame,
    y_test: pd.Series,
    max_rows: int = 15_000,
) -> pd.DataFrame:
    """Estimate final model feature importance with held-out permutation tests."""
    if len(x_test) > max_rows:
        x_importance = x_test.sample(n=max_rows, random_state=RANDOM_STATE)
        y_importance = y_test.loc[x_importance.index]
    else:
        x_importance = x_test
        y_importance = y_test

    result = permutation_importance(
        model,
        x_importance,
        y_importance,
        scoring="roc_auc",
        n_repeats=5,
        random_state=RANDOM_STATE,
        n_jobs=-1,
    )
    frame = pd.DataFrame(
        {
            "feature": list(x_importance.columns),
            "importance_mean": result.importances_mean,
```

```
            "importance_std": result.importances_std,
            "scoring": "roc_auc",
        }
    )
    return frame.sort_values("importance_mean", ascending=False)


def create_figures(
    df: pd.DataFrame,
    model: Pipeline,
    y_test: pd.Series,
    y_pred: np.ndarray,
    feature_importance: pd.DataFrame,
    report_dir: Path,
    initial_model: Pipeline | None = None,
) -> None:
    """Create EDA and model evaluation figures."""
    figure_dir = report_dir / "figures"
    figure_dir.mkdir(parents=True, exist_ok=True)
    sns.set_theme(style="whitegrid")

    plt.figure(figsize=(8, 5))
    sns.histplot(df["trip_duration_min"], bins=60, color="#2f6f73")
    plt.axvline(30, color="#c03a2b", linestyle="--", label="30-minute threshold")
    plt.title("Distribution of Cleaned Trip Duration")
    plt.xlabel("Trip duration (minutes)")
    plt.ylabel("Trips")
    plt.legend()
    plt.tight_layout()
    plt.savefig(figure_dir / "duration_distribution.png", dpi=180)
    plt.close()

    plot_sample = df.sample(n=min(len(df), 20000), random_state=RANDOM_STATE)
    plt.figure(figsize=(8, 5))
    sns.scatterplot(
        data=plot_sample,
        x="trip_distance",
        y="trip_duration_min",
        hue=TARGET,
        alpha=0.35,
        palette={0: "#5677a6", 1: "#d1683c"},
        linewidth=0,
    )
    plt.title("Trip Distance vs. Duration")
    plt.xlabel("Trip distance (miles)")
    plt.ylabel("Trip duration (minutes)")
    plt.tight_layout()
    plt.savefig(figure_dir / "distance_vs_duration.png", dpi=180)
    plt.close()

    hourly = (
        df.groupby("pickup_hour", as_index=False)[TARGET]
        .mean()
        .rename(columns={TARGET: "long_trip_rate"})
    )
    plt.figure(figsize=(8, 5))
    sns.lineplot(
        data=hourly,
        x="pickup_hour",
        y="long_trip_rate",
        marker="o",
        color="#234f68",
    )
    plt.title("Long-Trip Rate by Pickup Hour")
    plt.xlabel("Pickup hour")
    plt.ylabel("Share of trips lasting at least 30 minutes")
    plt.ylim(0, max(0.4, hourly["long_trip_rate"].max() + 0.03))
    plt.tight_layout()
    plt.savefig(figure_dir / "hourly_long_trip_rate.png", dpi=180)
    plt.close()

    cm = confusion_matrix(y_test, y_pred)
    display = ConfusionMatrixDisplay(
        confusion_matrix=cm,
        display_labels=["Under 30 min", "30+ min"],
    )
    display.plot(cmap="Blues", values_format="d")
    plt.title("Optimized Model Confusion Matrix")
    plt.tight_layout()
    plt.savefig(figure_dir / "confusion_matrix.png", dpi=180)
    plt.close()

    positive_importance = feature_importance[feature_importance["importance_mean"] > 0]
    importance_frame = positive_importance.head(10).sort_values(
        "importance_mean",
        ascending=True,
    )
    plt.figure(figsize=(9, 5.5))
    sns.barplot(
        data=importance_frame,
        y="feature",
        x="importance_mean",
        color="#2f6f73",
```

```
    )
    plt.title("Optimized Model Feature Importance")
    plt.xlabel("Mean ROC-AUC decrease after permutation")
    plt.ylabel("")
    plt.tight_layout()
    plt.savefig(figure_dir / "feature_importance.png", dpi=180)
    plt.close()

    if initial_model is None:
        return

    coefficient_frame = coefficient_summary(initial_model).head(15)
    plt.figure(figsize=(9, 6))
    sns.barplot(
        data=coefficient_frame,
        y="feature",
        x="coefficient",
        hue="direction",
        dodge=False,
        palette={
            "Higher long-trip risk": "#d1683c",
            "Lower long-trip risk": "#5677a6",
        },
    )
    plt.axvline(0, color="black", linewidth=0.8)
    plt.title("Initial Logistic Regression Coefficients")
    plt.xlabel("Standardized coefficient")
    plt.ylabel("")
    plt.legend(loc="lower right")
    plt.tight_layout()
    plt.savefig(figure_dir / "top_coefficients.png", dpi=180)
    plt.close()


def write_outputs(
    df: pd.DataFrame,
    raw_rows: int,
    clean_rows_before_sampling: int,
    preprocessing_audit: pd.DataFrame,
    feature_summary: pd.DataFrame,
    split_summary: dict[str, Any],
    baseline_metrics: dict[str, Any],
    initial_metrics: dict[str, Any],
    final_metrics: dict[str, Any],
    selected_candidate: dict[str, Any],
    final_threshold: float,
    tuning_results: pd.DataFrame,
    report: str,
    final_model: Pipeline,
    initial_model: Pipeline,
    feature_importance: pd.DataFrame,
    report_dir: Path,
    model_dir: Path,
) -> None:
    """Persist metrics, feature notes, and the trained model."""
    report_dir.mkdir(parents=True, exist_ok=True)
    model_dir.mkdir(parents=True, exist_ok=True)

    summary = {
        "dataset": {
            "source": DEFAULT_DATA_URL,
            "raw_rows": int(raw_rows),
            "clean_rows_before_sampling": int(clean_rows_before_sampling),
            "model_sample_rows": int(len(df)),
            "long_trip_rate": round(float(df[TARGET].mean()), 4),
            "median_duration_min": round(float(df["trip_duration_min"].median()), 2),
            "median_distance_miles": round(float(df["trip_distance"].median()), 2),
        },
        "target_distribution": {
            "under_30_min": int((df[TARGET] == 0).sum()),
            "long_30_plus_min": int((df[TARGET] == 1).sum()),
            "long_trip_rate": round(float(df[TARGET].mean()), 4),
        },
        "model_inputs": {
            "numeric_features": NUMERIC_FEATURES,
            "categorical_features": CATEGORICAL_FEATURES,
            "target": TARGET,
            "positive_class": "long_trip = 1 means trip duration is at least 30 minutes",
        },
        "train_test_split": split_summary,
        "model_optimization": {
            "selected_model": FINAL_MODEL_NAME,
            "selected_candidate": selected_candidate,
            "decision_threshold": round(float(final_threshold), 4),
            "selection_metric": (
                f"validation F0.5 with recall >= {DECISION_RECALL_FLOOR:.3f}"
            ),
            "reason": (
                "F0.5 emphasizes precision while the higher recall floor keeps "
                "long-trip recall above the Day 1 logistic reference."
            ),
        },
        "baseline_most_frequent": baseline_metrics,
```

```
        "initial_logistic_regression": initial_metrics,
        "logistic_regression": initial_metrics,
        "optimized_hist_gradient_boosting": final_metrics,
        "optimized_model": final_metrics,
        "classification_report": report,
    }

    with (report_dir / "metrics.json").open("w", encoding="utf-8") as file:
        json.dump(summary, file, indent=2)

    preprocessing_audit.to_csv(report_dir / "preprocessing_audit.csv", index=False)
    feature_summary.to_csv(report_dir / "feature_engineering_summary.csv", index=False)
    tuning_results.to_csv(report_dir / "tuning_results.csv", index=False)

    pd.DataFrame(
        [
            {"model": "Most frequent baseline", **baseline_metrics},
            {"model": INITIAL_MODEL_NAME, **initial_metrics},
            {"model": FINAL_MODEL_NAME, **final_metrics},
        ]
    ).to_csv(report_dir / "model_metrics.csv", index=False)

    pd.DataFrame(
        [
            {
                "target_value": 0,
                "label": "Under 30 minutes",
                "rows": int((df[TARGET] == 0).sum()),
                "share": round(float((df[TARGET] == 0).mean()), 4),
            },
            {
                "target_value": 1,
                "label": "30+ minutes",
                "rows": int((df[TARGET] == 1).sum()),
                "share": round(float((df[TARGET] == 1).mean()), 4),
            },
        ]
    ).to_csv(report_dir / "target_distribution.csv", index=False)

    coefficient_summary(initial_model).to_csv(
        report_dir / "model_coefficients.csv",
        index=False,
    )
    feature_importance.to_csv(report_dir / "model_feature_importance.csv", index=False)
    joblib.dump(final_model, model_dir / "long_trip_optimized_model.joblib")


def run_pipeline(config: PipelineConfig) -> dict[str, Any]:
    raw_path = download_file(config.data_url, config.raw_path, config.force_download)
    raw = load_raw_data(raw_path)
    raw_rows = len(raw)
    df, preprocessing_audit, feature_summary = preprocess_with_audit(raw)
    clean_rows = len(df)
    if config.sample_size and len(df) > config.sample_size:
        rows_before_sample = len(df)
        df = df.sample(n=config.sample_size, random_state=RANDOM_STATE)
        preprocessing_audit = pd.concat(
            [
                preprocessing_audit,
                pd.DataFrame(
                    [
                        {
                            "step": "13_model_sample",
                            "description": (
                                "Selected deterministic modeling sample with "
                                f"random_state={RANDOM_STATE}."
                            ),
                            "rows_before": rows_before_sample,
                            "rows_after": len(df),
                            "rows_removed": rows_before_sample - len(df),
                            "step_removed_pct": round(
                                ((rows_before_sample - len(df)) / rows_before_sample)
                                * 100,
                                4,
                            ),
                            "retained_pct_of_raw": round((len(df) / raw_rows) * 100, 4),
                        }
                    ]
                ),
            ],
            ignore_index=True,
        )

    x = df[FEATURES]
    y = df[TARGET]
    x_development, x_test, y_development, y_test = train_test_split(
        x,
        y,
        test_size=0.2,
        random_state=RANDOM_STATE,
        stratify=y,
    )
    x_train, x_validation, y_train, y_validation = train_test_split(
```

```
    x_development,
    y_development,
    test_size=0.25,
    random_state=RANDOM_STATE,
    stratify=y_development,
)
split_summary = {
    "split_method": "60/20/20 stratified train-validation-test split",
    "random_state": RANDOM_STATE,
    "train_rows": int(len(x_train)),
    "validation_rows": int(len(x_validation)),
    "test_rows": int(len(x_test)),
    "train_long_trip_rate": round(float(y_train.mean()), 4),
    "validation_long_trip_rate": round(float(y_validation.mean()), 4),
    "test_long_trip_rate": round(float(y_test.mean()), 4),
}

baseline = DummyClassifier(strategy="most_frequent")
baseline.fit(x_train, y_train)
baseline_pred = baseline.predict(x_test)
baseline_score = np.repeat(y_train.mean(), repeats=len(y_test))
baseline_metrics = evaluate_predictions(y_test, baseline_pred, baseline_score)

initial_model = make_logistic_model()
initial_model.fit(x_train, y_train)
initial_score = initial_model.predict_proba(x_test)[:, 1]
initial_pred = (initial_score >= 0.5).astype(int)
initial_metrics = evaluate_predictions(
    y_test,
    initial_pred,
    initial_score,
    threshold=0.5,
)

final_model, final_threshold, selected_candidate, tuning_results = (
    tune_hist_gradient_boosting(x_train, y_train, x_validation, y_validation)
)
y_score = final_model.predict_proba(x_test)[:, 1]
y_pred = (y_score >= final_threshold).astype(int)
final_metrics = evaluate_predictions(
    y_test,
    y_pred,
    y_score,
    threshold=final_threshold,
)

report = classification_report(
    y_test,
    y_pred,
    target_names=["Under 30 min", "30+ min"],
    digits=4,
)
feature_importance = feature_importance_summary(final_model, x_test, y_test)
create_figures(
    df,
    final_model,
    y_test,
    y_pred,
    feature_importance,
    config.report_dir,
    initial_model=initial_model,
)
write_outputs(
    df,
    raw_rows,
    clean_rows,
    preprocessing_audit,
    feature_summary,
    split_summary,
    baseline_metrics,
    initial_metrics,
    final_metrics,
    selected_candidate,
    final_threshold,
    tuning_results,
    report,
    final_model,
    initial_model,
    feature_importance,
    config.report_dir,
    config.model_dir,
)
return {
    "raw_rows": raw_rows,
    "clean_rows_before_sampling": clean_rows,
    "sample_rows": len(df),
    "long_trip_rate": round(float(df[TARGET].mean()), 4),
    "initial_metrics": initial_metrics,
    "optimized_metrics": final_metrics,
    "baseline_metrics": baseline_metrics,
    "selected_candidate": selected_candidate["candidate"],
    "decision_threshold": round(float(final_threshold), 4),
}
```

```
def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--data-url", default=DEFAULT_DATA_URL)
    parser.add_argument("--raw-path", type=Path, default=DEFAULT_RAW_PATH)
    parser.add_argument("--report-dir", type=Path, default=DEFAULT_REPORT_DIR)
    parser.add_argument("--model-dir", type=Path, default=DEFAULT_MODEL_DIR)
    parser.add_argument("--sample-size", type=int, default=120_000)
    parser.add_argument("--force-download", action="store_true")
    return parser.parse_args()


def main() -> None:
    args = parse_args()
    config = PipelineConfig(
        data_url=args.data_url,
        raw_path=args.raw_path,
        report_dir=args.report_dir,
        model_dir=args.model_dir,
        sample_size=args.sample_size,
        force_download=args.force_download,
    )
    result = run_pipeline(config)
    print(json.dumps(result, indent=2, default=str))


if __name__ == "__main__":
    main()
```

Appendix C: Prediction Notebook Code Cells

Prediction notebook code cell 1

```
from pathlib import Path
import os
import shutil
import subprocess
import sys

repo_url = "https://github.com/soumithkumara/project_day_1.git"
repo_branch = "project_day_2"
repo_dir = Path("/content/project_day_1_project_day_2")

if repo_dir.exists() and (repo_dir / ".git").exists():
    subprocess.run(["git", "-C", str(repo_dir), "fetch", "origin", repo_branch], check=True)
    subprocess.run(["git", "-C", str(repo_dir), "reset", "--hard", f"origin/{repo_branch}"],
        check=True)
else:
    if repo_dir.exists():
        shutil.rmtree(repo_dir)
    subprocess.run(
        ["git", "clone", "--branch", repo_branch, "--single-branch", repo_url, str(repo_dir)],
        check=True,
    )

os.chdir(repo_dir)
active_branch = subprocess.check_output(["git", "branch", "--show-current"], text=True).strip()
active_commit = subprocess.check_output(["git", "rev-parse", "--short", "HEAD"],
    text=True).strip()
print(f"Working directory: {Path.cwd()}")
print(f"Active branch: {active_branch}")
print(f"Active commit: {active_commit}")
print("Repo files:", sorted(path.name for path in Path.cwd().iterdir()))
```

Prediction notebook code cell 2

```
import importlib

required_packages = {
    "pandas": "pandas",
    "numpy": "numpy",
    "sklearn": "scikit-learn",
    "joblib": "joblib",
    "pyarrow": "pyarrow",
    "matplotlib": "matplotlib",
    "seaborn": "seaborn",
}

missing_packages = []
for module_name, package_name in required_packages.items():
    try:
        importlib.import_module(module_name)
    except ModuleNotFoundError:
        missing_packages.append(package_name)
    except Exception as exc:
        raise RuntimeError(
            "The Colab runtime has a partially upgraded or inconsistent Python package stack. "
            "Choose Runtime > Restart session, then run this notebook from the top."
        ) from exc

if missing_packages:
    print("Installing missing packages:", missing_packages)
    subprocess.run([sys.executable, "-m", "pip", "install", *missing_packages], check=True)
    print("If Colab asks you to restart the runtime, restart and then run all cells again.")
else:
    print("All required packages are already available. No package reinstall needed.")
```

Prediction notebook code cell 3

```
model_path = Path("models/long_trip_optimized_model.joblib")
metrics_path = Path("reports/metrics.json")

if not model_path.exists() or not metrics_path.exists():
    print("Optimized model or metrics not found. Running the training pipeline first...")
    subprocess.run([sys.executable, "src/nyc_taxi_long_trip_model.py"], check=True)
else:
    print(f"Using existing optimized model: {model_path}")

print(f"Model ready: {model_path.exists()}")
print(f"Metrics ready: {metrics_path.exists()}")
```

Prediction notebook code cell 4

```
import json

import joblib
import pandas as pd
```

```
model = joblib.load(model_path)
metrics = json.loads(metrics_path.read_text())
DECISION_THRESHOLD = metrics["model_optimization"]["decision_threshold"]
print(f"Using tuned long-trip threshold: {DECISION_THRESHOLD:.2f}")

# TLC location IDs for Newark, JFK, and LaGuardia airport zones.
AIRPORT_ZONE_IDS = {1, 132, 138}

def build_trip_features(
    pickup_datetime,
    trip_distance,
    passenger_count,
    vendor_id,
    ratecode_id,
    pu_location_id,
    do_location_id,
):
    pickup_datetime = pd.to_datetime(pickup_datetime)

    return pd.DataFrame([{
        "trip_distance": float(trip_distance),
        "passenger_count": int(passenger_count),
        "pickup_hour": pickup_datetime.hour,
        "pickup_dayofweek": pickup_datetime.dayofweek,
        "is_weekend": int(pickup_datetime.dayofweek in [5, 6]),
        "airport_trip": int(
            int(pu_location_id) in AIRPORT_ZONE_IDS
            or int(do_location_id) in AIRPORT_ZONE_IDS
        ),
        "VendorID": str(vendor_id),
        "RatecodeID": str(ratecode_id),
        "PULocationID": str(pu_location_id),
        "DOLocationID": str(do_location_id),
    }])

def predict_long_trip(
    pickup_datetime,
    trip_distance,
    passenger_count,
    vendor_id,
    ratecode_id,
    pu_location_id,
    do_location_id,
):
    features = build_trip_features(
        pickup_datetime=pickup_datetime,
        trip_distance=trip_distance,
        passenger_count=passenger_count,
        vendor_id=vendor_id,
        ratecode_id=ratecode_id,
        pu_location_id=pu_location_id,
        do_location_id=do_location_id,
    )

    probability = float(model.predict_proba(features)[0][1])
    prediction = int(probability >= DECISION_THRESHOLD)

    return features, pd.DataFrame([{
        "prediction_label": "Long trip: 30 minutes or more" if prediction == 1 else "Short trip:
            under 30 minutes",
        "predicted_long_trip_class": prediction,
        "long_trip_probability": round(probability, 4),
        "long_trip_probability_percent": f"{probability:.2%}",
        "decision_threshold": DECISION_THRESHOLD,
    }])
```

Prediction notebook code cell 5

```
# @title New Taxi Trip Input
pickup_datetime = "2024-01-15 17:30:00"  # @param {type:"string"}
trip_distance = 14.2  # @param {type:"number"}
passenger_count = 2  # @param {type:"integer"}
vendor_id = 2  # @param {type:"integer"}
ratecode_id = 1  # @param {type:"integer"}
pu_location_id = 132  # @param {type:"integer"}
do_location_id = 48  # @param {type:"integer"}

features, prediction_result = predict_long_trip(
    pickup_datetime=pickup_datetime,
    trip_distance=trip_distance,
    passenger_count=passenger_count,
    vendor_id=vendor_id,
    ratecode_id=ratecode_id,
    pu_location_id=pu_location_id,
    do_location_id=do_location_id,
)

print("Model input features:")
display(features)

print("Prediction:")
display(prediction_result)
```

Prediction notebook code cell 6

```
new_trips = [
    {
        "pickup_datetime": "2024-01-15 17:30:00",
        "trip_distance": 14.2,
        "passenger_count": 2,
        "vendor_id": 2,
        "ratecode_id": 1,
        "pu_location_id": 132,
        "do_location_id": 48,
    },
    {
        "pickup_datetime": "2024-01-16 10:15:00",
        "trip_distance": 1.4,
        "passenger_count": 1,
        "vendor_id": 2,
        "ratecode_id": 1,
        "pu_location_id": 237,
        "do_location_id": 236,
    },
]

batch_results = []
for i, trip in enumerate(new_trips, start=1):
    _, result = predict_long_trip(**trip)
    row = {"trip_number": i, **trip, **result.iloc[0].to_dict()}
    batch_results.append(row)

display(pd.DataFrame(batch_results))
```

Appendix D: Day 2 Colab Notebook Code Cells

Day 2 notebook code cell 1

```
from pathlib import Path
import os
import shutil
import subprocess

repo_url = "https://github.com/soumithkumara/project_day_1.git"
repo_branch = "project_day_2"
repo_dir = Path("/content/project_day_1_project_day_2")

if repo_dir.exists() and (repo_dir / ".git").exists():
    subprocess.run(["git", "-C", str(repo_dir), "fetch", "origin", repo_branch], check=True)
    subprocess.run(["git", "-C", str(repo_dir), "reset", "--hard", f"origin/{repo_branch}"],
        check=True)
else:
    if repo_dir.exists():
        shutil.rmtree(repo_dir)
    subprocess.run(
        ["git", "clone", "--branch", repo_branch, "--single-branch", repo_url, str(repo_dir)],
        check=True,
    )

os.chdir(repo_dir)
active_branch = subprocess.check_output(["git", "branch", "--show-current"], text=True).strip()
active_commit = subprocess.check_output(["git", "rev-parse", "--short", "HEAD"],
    text=True).strip()
print(f"Working directory: {Path.cwd()}")
print(f"Active branch: {active_branch}")
print(f"Active commit: {active_commit}")
print("Repo files:", sorted(path.name for path in Path.cwd().iterdir()))
```

Day 2 notebook code cell 2

```
!pip install -r requirements.txt
```

Day 2 notebook code cell 3

```
from src.nyc_taxi_long_trip_model import (
    DEFAULT_DATA_URL,
    DEFAULT_MODEL_DIR,
    DEFAULT_RAW_PATH,
    DEFAULT_REPORT_DIR,
    PipelineConfig,
    run_pipeline,
)

config = PipelineConfig(
    data_url=DEFAULT_DATA_URL,
    raw_path=DEFAULT_RAW_PATH,
    report_dir=DEFAULT_REPORT_DIR,
    model_dir=DEFAULT_MODEL_DIR,
    sample_size=120_000,
    force_download=False,
)

result = run_pipeline(config)
result
```

Day 2 notebook code cell 4

```
import pandas as pd
from IPython.display import Markdown, display

audit = pd.read_csv("reports/preprocessing_audit.csv")
display(audit)

raw_rows = int(audit.loc[audit["step"] == "01_raw_load", "rows_after"].iloc[0])
clean_rows = int(audit.loc[audit["step"] == "12_feature_engineering", "rows_after"].iloc[0])
sample_rows = int(audit.loc[audit["step"] == "13_model_sample", "rows_after"].iloc[0])
display(Markdown(
    f"**Summary:** The raw file started with **{raw_rows:,} rows**. "
    f"After cleaning, **{clean_rows:,} realistic trips** remained. "
    f"The model used a deterministic **{sample_rows:,}-row sample** so the notebook runs quickly
        and reproducibly."
))
```

Day 2 notebook code cell 5

```
feature_changes = pd.read_csv("reports/feature_engineering_summary.csv")
display(feature_changes)

target_distribution = pd.read_csv("reports/target_distribution.csv")
display(Markdown("### Target Distribution After Preprocessing"))
display(target_distribution)
```

Day 2 notebook code cell 6

```
import json
from pathlib import Path

metrics_path = Path("reports/metrics.json")
metrics = json.loads(metrics_path.read_text())
print(json.dumps(metrics, indent=2))

display(Markdown("### Classification Report"))
print(metrics["classification_report"])
```

Day 2 notebook code cell 7

```
import pandas as pd
from IPython.display import display

display(pd.read_csv("reports/model_metrics.csv"))
display(pd.read_csv("reports/tuning_results.csv"))
display(pd.read_csv("reports/model_feature_importance.csv"))
```

Day 2 notebook code cell 8

```
from pathlib import Path
from IPython.display import Image, Markdown, display

figure_dir = Path("reports/figures")
for figure_path in sorted(figure_dir.glob("*.png")):
    title = figure_path.stem.replace("_", " ").title()
    display(Markdown(f"### {title}"))
    display(Image(filename=str(figure_path)))
```

Appendix E: PDF Generation Script

This appendix includes the script that generated this one-file submission PDF so the packet itself is reproducible.

```
from __future__ import annotations

import csv
import html
import json
import textwrap
from pathlib import Path

from reportlab.lib import colors
from reportlab.lib.enums import TA_CENTER, TA_LEFT, TA_RIGHT
from reportlab.lib.pagesizes import letter
from reportlab.lib.styles import ParagraphStyle, getSampleStyleSheet
from reportlab.lib.units import inch
from reportlab.platypus import (
    Image,
    KeepTogether,
    PageBreak,
    Paragraph,
    Preformatted,
    SimpleDocTemplate,
    Spacer,
    Table,
    TableStyle,
)


ROOT = Path(__file__).resolve().parents[1]
REPORTS = ROOT / "reports"
FIGURES = REPORTS / "figures"
SRC = ROOT / "src"
SCRIPTS = ROOT / "scripts"
NOTEBOOKS = ROOT / "notebooks"
OUTPUT_DIR = ROOT / "output" / "pdf"
OUTPUT_PDF = OUTPUT_DIR / "NYC_Taxi_Deliverable_2_Full_Submission_Report.pdf"

INK = colors.HexColor("#111111")
DARK_GRAY = colors.HexColor("#333333")
MID_GRAY = colors.HexColor("#666666")
LIGHT_GRAY = colors.HexColor("#F1F1F1")
LINE_GRAY = colors.HexColor("#B8B8B8")
SOFT_GRAY = colors.HexColor("#FAFAFA")


def load_json(path: Path) -> dict:
    return json.loads(path.read_text(encoding="utf-8"))


def load_csv(path: Path) -> list[dict[str, str]]:
    with path.open(newline="", encoding="utf-8") as file:
        return list(csv.DictReader(file))


def read_text(path: Path) -> str:
    return path.read_text(encoding="utf-8")


def safe(text: object) -> str:
    return html.escape(str(text), quote=False)


def para(text: str, style: ParagraphStyle) -> Paragraph:
    return Paragraph(safe(text), style)


def bullet(text: str, style: ParagraphStyle) -> Paragraph:
    return Paragraph(f"- {safe(text)}", style)


def add_bullets(story: list, items: list[str], style: ParagraphStyle) -> None:
    for item in items:
        story.append(bullet(item, style))


def code_block(text: str, style: ParagraphStyle, width: int = 95) -> Preformatted:
    wrapped_lines: list[str] = []
    for raw_line in text.splitlines():
        line = raw_line.rstrip()
        if len(line) <= width:
            wrapped_lines.append(line)
            continue
        indent = len(line) - len(line.lstrip(" "))
        subsequent = " " * min(indent + 4, 16)
        wrapped_lines.extend(
            textwrap.wrap(
                line,
                width=width,
                subsequent_indent=subsequent,
```

```
                replace_whitespace=False,
                drop_whitespace=False,
                break_long_words=False,
                break_on_hyphens=False,
            )
        )
    return Preformatted("\n".join(wrapped_lines), style)


def notebook_code_cells(path: Path) -> list[str]:
    notebook = load_json(path)
    cells: list[str] = []
    for cell in notebook.get("cells", []):
        if cell.get("cell_type") == "code":
            source = "".join(cell.get("source", []))
            if source.strip():
                cells.append(source)
    return cells


def metric(metrics: dict, key: str) -> str:
    return f"{metrics[key]:.4f}"


def pct(metrics: dict, key: str) -> str:
    return f"{metrics[key] * 100:.2f}%"


def metric_change(new: float, old: float) -> str:
    return f"{(new - old) * 100:+.2f} pts"


def format_class_weight(value: object) -> str:
    if not isinstance(value, dict):
        return str(value)
    try:
        items = sorted(value.items(), key=lambda item: int(item[0]))
    except (TypeError, ValueError):
        items = sorted(value.items(), key=lambda item: str(item[0]))
    return "{" + ", ".join(f"{key}: {weight}" for key, weight in items) + "}"


def safe_divide(numerator: float, denominator: float) -> float:
    return numerator / denominator if denominator else 0.0


def binary_f1(precision: float, recall: float) -> float:
    return safe_divide(2 * precision * recall, precision + recall)


def classification_summary_rows(final: dict) -> list[list[str]]:
    tn = final["true_negative"]
    fp = final["false_positive"]
    fn = final["false_negative"]
    tp = final["true_positive"]
    support_under = tn + fp
    support_long = tp + fn
    total = support_under + support_long

    under_precision = safe_divide(tn, tn + fn)
    under_recall = safe_divide(tn, tn + fp)
    under_f1 = binary_f1(under_precision, under_recall)
    long_precision = final["precision"]
    long_recall = final["recall"]
    long_f1 = final["f1"]
    weighted_precision = safe_divide(
        under_precision * support_under + long_precision * support_long,
        total,
    )
    weighted_recall = safe_divide(
        under_recall * support_under + long_recall * support_long,
        total,
    )
    weighted_f1 = safe_divide(
        under_f1 * support_under + long_f1 * support_long,
        total,
    )

    return [
        ["Class", "Precision", "Recall", "F1-score", "Support"],
        ["Under 30 min", f"{under_precision:.4f}", f"{under_recall:.4f}", f"{under_f1:.4f}",
            f"{support_under:,}"],
        ["30+ min", f"{long_precision:.4f}", f"{long_recall:.4f}", f"{long_f1:.4f}",
            f"{support_long:,}"],
        ["Weighted avg", f"{weighted_precision:.4f}", f"{weighted_recall:.4f}",
            f"{weighted_f1:.4f}", f"{total:,}"],
    ]


def make_table(
    data: list[list[str]],
    widths: list[float],
    font_size: float = 8.2,
```

```
    header: bool = True,
) -> Table:
    table = Table(data, colWidths=[w * inch for w in widths], repeatRows=1 if header else 0)
    commands = [
        ("FONTNAME", (0, 0), (-1, -1), "Helvetica"),
        ("FONTSIZE", (0, 0), (-1, -1), font_size),
        ("TEXTCOLOR", (0, 0), (-1, -1), INK),
        ("GRID", (0, 0), (-1, -1), 0.35, LINE_GRAY),
        ("VALIGN", (0, 0), (-1, -1), "TOP"),
        ("LEFTPADDING", (0, 0), (-1, -1), 5),
        ("RIGHTPADDING", (0, 0), (-1, -1), 5),
        ("TOPPADDING", (0, 0), (-1, -1), 4),
        ("BOTTOMPADDING", (0, 0), (-1, -1), 4),
        ("ROWBACKGROUNDS", (0, 1), (-1, -1), [colors.white, SOFT_GRAY]),
    ]
    if header:
        commands.extend(
            [
                ("FONTNAME", (0, 0), (-1, 0), "Helvetica-Bold"),
                ("BACKGROUND", (0, 0), (-1, 0), LIGHT_GRAY),
                ("ALIGN", (1, 1), (-1, -1), "RIGHT"),
            ]
        )
    table.setStyle(TableStyle(commands))
    return table


def add_captioned_figure(
    story: list,
    image_path: Path,
    caption: str,
    styles,
    width: float = 6.0,
    height: float = 3.75,
) -> None:
    if not image_path.exists():
        return
    story.append(
        KeepTogether(
            [
                para(caption, styles["FigureCaption"]),
                Image(str(image_path), width=width * inch, height=height * inch),
            ]
        )
    )
    story.append(Spacer(1, 0.16 * inch))


def page_footer(canvas, doc):
    canvas.saveState()
    canvas.setStrokeColor(LINE_GRAY)
    canvas.setLineWidth(0.4)
    canvas.line(0.72 * inch, 0.58 * inch, 7.78 * inch, 0.58 * inch)
    canvas.setFont("Helvetica", 8)
    canvas.setFillColor(MID_GRAY)
    canvas.drawString(0.72 * inch, 0.42 * inch, "NYC Taxi Deliverable 2 Full Submission Report")
    canvas.drawRightString(7.78 * inch, 0.42 * inch, f"Page {doc.page}")
    canvas.restoreState()


def build_styles():
    styles = getSampleStyleSheet()
    styles.add(
        ParagraphStyle(
            name="CoverTitle",
            parent=styles["Title"],
            fontName="Helvetica-Bold",
            fontSize=21,
            leading=26,
            alignment=TA_CENTER,
            textColor=INK,
            spaceAfter=12,
        )
    )
    styles.add(
        ParagraphStyle(
            name="CoverSubtitle",
            parent=styles["Normal"],
            fontName="Helvetica",
            fontSize=11,
            leading=15,
            alignment=TA_CENTER,
            textColor=DARK_GRAY,
            spaceAfter=8,
        )
    )
    styles.add(
        ParagraphStyle(
            name="Section",
            parent=styles["Heading1"],
            fontName="Helvetica-Bold",
            fontSize=14,
            leading=17,
```

```
            textColor=INK,
            spaceBefore=14,
            spaceAfter=7,
        )
    )
    styles.add(
        ParagraphStyle(
            name="Subsection",
            parent=styles["Heading2"],
            fontName="Helvetica-Bold",
            fontSize=11,
            leading=14,
            textColor=INK,
            spaceBefore=9,
            spaceAfter=5,
        )
    )
    styles.add(
        ParagraphStyle(
            name="Body",
            parent=styles["BodyText"],
            fontName="Helvetica",
            fontSize=9.6,
            leading=13.2,
            alignment=TA_LEFT,
            textColor=INK,
            spaceAfter=6,
        )
    )
    styles.add(
        ParagraphStyle(
            name="BulletCustom",
            parent=styles["Body"],
            leftIndent=14,
            firstLineIndent=-9,
            spaceAfter=4,
        )
    )
    styles.add(
        ParagraphStyle(
            name="Small",
            parent=styles["Body"],
            fontSize=8.4,
            leading=11.2,
            textColor=DARK_GRAY,
        )
    )
    styles.add(
        ParagraphStyle(
            name="FigureCaption",
            parent=styles["Body"],
            fontName="Helvetica-Bold",
            fontSize=8.6,
            leading=11,
            alignment=TA_CENTER,
            textColor=INK,
            spaceAfter=4,
        )
    )
    styles.add(
        ParagraphStyle(
            name="CodeBlockCustom",
            parent=styles["Code"],
            fontName="Courier",
            fontSize=6.1,
            leading=7.2,
            borderColor=LINE_GRAY,
            borderWidth=0.4,
            borderPadding=5,
            backColor=SOFT_GRAY,
            textColor=INK,
            spaceBefore=3,
            spaceAfter=7,
        )
    )
    styles.add(
        ParagraphStyle(
            name="TOCLine",
            parent=styles["Body"],
            fontName="Helvetica",
            fontSize=9.8,
            leading=13,
            spaceAfter=3,
        )
    )
    styles.add(
        ParagraphStyle(
            name="RightSmall",
            parent=styles["Small"],
            alignment=TA_RIGHT,
        )
    )
    return styles
```

```
def add_markdown_report(story: list, styles) -> None:
    """Render the assignment markdown in a simple, controlled way."""
    for line in read_text(REPORTS / "assignment_report.md").splitlines():
        stripped = line.strip()
        if not stripped:
            continue
        if stripped.startswith("# "):
            story.append(para(stripped[2:], styles["Section"]))
        elif stripped.startswith("## "):
            story.append(para(stripped[3:], styles["Section"]))
        elif stripped.startswith("- "):
            story.append(bullet(stripped[2:], styles["BulletCustom"]))
        elif stripped.startswith("|"):
            continue
        else:
            clean = stripped.replace("**", "").replace("`", "")
            story.append(para(clean, styles["Body"]))


def build_pdf() -> Path:
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
    metrics = load_json(REPORTS / "metrics.json")
    model_rows = load_csv(REPORTS / "model_metrics.csv")
    tuning_rows = load_csv(REPORTS / "tuning_results.csv")
    feature_importance = load_csv(REPORTS / "model_feature_importance.csv")
    audit = load_csv(REPORTS / "preprocessing_audit.csv")
    feature_summary = load_csv(REPORTS / "feature_engineering_summary.csv")
    requirements = read_text(ROOT / "requirements.txt").strip()
    main_code = read_text(SRC / "nyc_taxi_long_trip_model.py")
    pdf_script = read_text(SCRIPTS / "create_full_submission_pdf.py")
    prediction_cells = notebook_code_cells(NOTEBOOKS / "predict_new_taxi_trip.ipynb")
    day2_cells = notebook_code_cells(NOTEBOOKS / "nyc_taxi_long_trip_model_day2.ipynb")

    baseline = metrics["baseline_most_frequent"]
    initial = metrics["initial_logistic_regression"]
    final = metrics["optimized_hist_gradient_boosting"]
    selected = metrics["model_optimization"]["selected_candidate"]
    class_weight_text = format_class_weight(selected["class_weight"])

    styles = build_styles()
    story: list = []

    story.append(Spacer(1, 0.55 * inch))
    story.append(para("NYC Taxi AI Analytics Project", styles["CoverTitle"]))
    story.append(
        para(
            "Deliverable 2 Full Submission Report: Model Optimization, Metrics, Ethics, Outputs,
                and Code",
            styles["CoverSubtitle"],
        )
    )
    story.append(Spacer(1, 0.15 * inch))
    story.append(
        make_table(
            [
                ["Field", "Value"],
                ["Repository branch", "project_day_2"],
                ["Project objective", "Predict whether a cleaned NYC yellow taxi trip lasts at
                least 30 minutes"],
                ["Final model", "Optimized histogram gradient boosting classifier"],
                ["Final test accuracy", pct(final, "accuracy")],
                ["Final test precision", pct(final, "precision")],
                ["Team members", "Soumith Kumar Ananthula, Varun Reddy Beesam, Vivek
                Doppalapudi"],
            ],
            [1.65, 4.85],
            font_size=8.7,
        )
    )
    story.append(Spacer(1, 0.2 * inch))
    story.append(
        para(
            "This PDF is designed as a one-file submission packet. It includes the written
                deliverable, dataset and preprocessing summary, hyperparameter tuning details,
                final evaluation metrics, figures, run instructions, dependencies, generated
                output inventory, and core source code.",
            styles["Body"],
        )
    )

    story.append(PageBreak())
    story.append(para("Submission Contents", styles["Section"]))
    contents = [
        "1. Executive summary",
        "2. Required Deliverable 2 report sections",
        "3. Dataset, preprocessing, and feature engineering evidence",
        "4. Hyperparameter tuning details and selected configuration",
        "5. Final model metrics and metric changes",
        "6. Key generated figures",
        "7. Reproducibility guide for Google Colab and local Python",
        "8. Generated artifacts inventory",
        "9. Appendix A: requirements.txt",
        "10. Appendix B: main training pipeline source code",
```

```
    "11. Appendix C: prediction notebook code cells",
    "12. Appendix D: Day 2 Colab notebook setup and pipeline cells",
    "13. Appendix E: PDF generation script",
]
for item in contents:
    story.append(para(item, styles["TOCLine"]))

story.append(PageBreak())
story.append(para("1. Executive Summary", styles["Section"]))
story.append(
    para(
        "The Day 2 work refined the Day 1 taxi long-trip classifier by adding a validation
            split, testing histogram gradient boosting candidates, tuning model
            hyperparameters, and selecting a recall-protected probability threshold. The final
             model predicts whether a trip is likely to last 30 minutes or longer.",
        styles["Body"],
    )
)
story.append(
    para(
        f"Compared with the Day 1 logistic regression model, the optimized model increased
            accuracy from {metric(initial, 'accuracy')} to {metric(final, 'accuracy')},
            precision from {metric(initial, 'precision')} to {metric(final, 'precision')}, and
             recall from {metric(initial, 'recall')} to {metric(final, 'recall')}. F1 improved
             from {metric(initial, 'f1')} to {metric(final, 'f1')}, and ROC-AUC improved from
            {metric(initial, 'roc_auc')} to {metric(final, 'roc_auc')}. The final threshold
            was lowered from the earlier strict version so long-trip recall stays above the
            logistic reference.",
        styles["Body"],
    )
)
add_bullets(
    story,
    [
        "Primary gain: fewer false long-trip alerts while preserving long-trip recall.",
        f"False positives dropped from {initial['false_positive']:,} to
            {final['false_positive']:,}.",
        f"False negatives dropped from {initial['false_negative']:,} to
            {final['false_negative']:,}.",
        "Recommended use: aggregate planning and decision support, not automated ride denial
            or neighborhood-level service restriction.",
    ],
    styles["BulletCustom"],
)

story.append(para("2. Deliverable 2 Written Report", styles["Section"]))
add_markdown_report(story, styles)

story.append(PageBreak())
story.append(para("3. Dataset and Preprocessing Evidence", styles["Section"]))
story.append(
    para(
        f"The pipeline used the official NYC TLC January 2024 yellow taxi parquet file. It
            loaded {metrics['dataset']['raw_rows']:,} raw rows, retained
            {metrics['dataset']['clean_rows_before_sampling']:,} realistic rows after
            cleaning, and used a deterministic {metrics['dataset']['model_sample_rows']:,}-row
             sample for modeling.",
        styles["Body"],
    )
)
selected_steps = {
    "03_filter_pickup_month",
    "04_filter_duration",
    "05_filter_distance",
    "06_filter_passengers",
    "07_filter_positive_fare",
    "09_filter_standard_ratecode",
    "13_model_sample",
}
audit_table = [["Preprocessing step", "Rows removed", "Rows after"]]
for row in audit:
    if row["step"] in selected_steps:
        audit_table.append(
            [
                row["description"],
                f"{int(row['rows_removed']):,}",
                f"{int(row['rows_after']):,}",
            ]
        )
story.append(make_table(audit_table, [4.35, 1.05, 1.1], font_size=7.3))

story.append(para("Feature Engineering Summary", styles["Subsection"]))
feature_table = [["Feature", "Type", "Purpose"]]
for row in feature_summary:
    feature_table.append([row["column"], row["type"], row["purpose"]])
story.append(make_table(feature_table, [1.45, 1.25, 3.8], font_size=7.0))

story.append(PageBreak())
story.append(para("4. Hyperparameter Tuning Details", styles["Section"]))
story.append(
    para(
        f"The selected model was {metrics['model_optimization']['selected_model']}. Five
            histogram-gradient-boosting candidates were tested on the validation set. The
```

```
            selected candidate was {selected['candidate']} with max_iter =
            {selected['max_iter']}, learning_rate = {selected['learning_rate']},
            max_leaf_nodes = {selected['max_leaf_nodes']}, l2_regularization =
            {selected['l2_regularization']}, class_weight = {class_weight_text}, and a
            decision threshold of {metrics['model_optimization']['decision_threshold']:.2f}.",
        styles["Body"],
    )
)
story.append(
    para(
        f"The selection metric was {metrics['model_optimization']['selection_metric']}. F0.5
            gives more weight to precision than recall, which matches the assignment request
            to make the model more accurate and precise, while the higher recall floor keeps
            long-trip recall above the Day 1 logistic reference.",
        styles["Body"],
    )
)
tuning_table = [[
    "Candidate",
    "Threshold",
    "Max iter",
    "Learning rate",
    "Leaves",
    "L2",
    "Val precision",
    "Val recall",
    "Selected",
]]
for row in tuning_rows:
    tuning_table.append(
        [
            row["candidate"],
            f"{float(row['selected_threshold']):.2f}",
            row["max_iter"],
            row["learning_rate"],
            row["max_leaf_nodes"],
            row["l2_regularization"],
            f"{float(row['validation_precision']):.4f}",
            f"{float(row['validation_recall']):.4f}",
            row["selected"],
        ]
    )
story.append(make_table(tuning_table, [1.35, 0.62, 0.62, 0.78, 0.55, 0.45, 0.78, 0.68, 0.67],
    font_size=6.6))

story.append(para("5. Final Model Metrics and Changes", styles["Section"]))
model_table = [["Model", "Accuracy", "Precision", "Recall", "F1", "F0.5", "ROC-AUC"]]
for row in model_rows:
    model_table.append(
        [
            row["model"],
            f"{float(row['accuracy']):.4f}",
            f"{float(row['precision']):.4f}",
            f"{float(row['recall']):.4f}",
            f"{float(row['f1']):.4f}",
            f"{float(row['f0_5']):.4f}",
            f"{float(row['roc_auc']):.4f}",
        ]
    )
story.append(make_table(model_table, [1.9, 0.72, 0.78, 0.72, 0.62, 0.62, 0.72],
    font_size=7.5))
impact_table = [
    ["Metric", "Day 1 logistic", "Day 2 optimized", "Change"],
    ["Accuracy", metric(initial, "accuracy"), metric(final, "accuracy"),
        metric_change(final["accuracy"], initial["accuracy"])],
    ["Precision", metric(initial, "precision"), metric(final, "precision"),
        metric_change(final["precision"], initial["precision"])],
    ["Recall", metric(initial, "recall"), metric(final, "recall"),
        metric_change(final["recall"], initial["recall"])],
    ["F1", metric(initial, "f1"), metric(final, "f1"), metric_change(final["f1"],
        initial["f1"])],
    ["ROC-AUC", metric(initial, "roc_auc"), metric(final, "roc_auc"),
        metric_change(final["roc_auc"], initial["roc_auc"])],
    ["False positives", f"{initial['false_positive']:,}", f"{final['false_positive']:,}",
        f"{final['false_positive'] - initial['false_positive']:,}"],
    ["False negatives", f"{initial['false_negative']:,}", f"{final['false_negative']:,}",
        f"{final['false_negative'] - initial['false_negative']:,}"],
]
story.append(make_table(impact_table, [1.55, 1.25, 1.35, 1.25], font_size=7.8))
story.append(para("Final classification report", styles["Subsection"]))
classification_table = classification_summary_rows(final)
story.append(make_table(classification_table, [1.6, 0.95, 0.95, 0.95, 0.95], font_size=8.0))

story.append(PageBreak())
story.append(para("6. Figures and Model Interpretation", styles["Section"]))
add_captioned_figure(story, FIGURES / "duration_distribution.png", "Figure 1. Cleaned trip
    duration distribution with the 30-minute threshold.", styles, 5.9, 3.55)
add_captioned_figure(story, FIGURES / "hourly_long_trip_rate.png", "Figure 2. Long-trip rate
    by pickup hour.", styles, 5.9, 3.55)
story.append(PageBreak())
add_captioned_figure(story, FIGURES / "confusion_matrix.png", "Figure 3. Optimized model
    confusion matrix on the held-out test set.", styles, 5.0, 4.0)
add_captioned_figure(story, FIGURES / "feature_importance.png", "Figure 4. Permutation feature
```

```
         importance for the optimized model.", styles, 5.9, 3.55)
    story.append(PageBreak())
    story.append(para("Feature importance values", styles["Subsection"]))
    importance_table = [["Feature", "Importance mean", "Importance std", "Scoring"]]
    for row in feature_importance[:10]:
        importance_table.append(
            [
                row["feature"],
                f"{float(row['importance_mean']):.6f}",
                f"{float(row['importance_std']):.6f}",
                row["scoring"],
            ]
        )
    story.append(make_table(importance_table, [2.2, 1.35, 1.25, 1.0], font_size=7.5))

    story.append(PageBreak())
    story.append(para("7. Reproducibility Guide", styles["Section"]))
    story.append(para("Google Colab setup", styles["Subsection"]))
    story.append(
        code_block(
            """!git clone --branch project_day_2
                https://github.com/soumithkumara/project_day_1.git
%cd project_day_1
!pip install -r requirements.txt
!python src/nyc_taxi_long_trip_model.py""",
            styles["CodeBlockCustom"],
            width=92,
        )
    )
    story.append(para("Recommended notebook", styles["Subsection"]))
    story.append(
        para(
            "Open notebooks/nyc_taxi_long_trip_model_day2.ipynb for a fresh Colab run. The setup
                cell forces the project_day_2 branch and prints the active branch and commit.",
            styles["Body"],
        )
    )
    story.append(para("Local run", styles["Subsection"]))
    story.append(
        code_block(
            """.\\.venv\\Scripts\\python.exe -m pip install -r requirements.txt
.\\.venv\\Scripts\\python.exe src\\nyc_taxi_long_trip_model.py""",
            styles["CodeBlockCustom"],
            width=92,
        )
    )
    story.append(para("Generated artifact inventory", styles["Subsection"]))
    inventory = [
        ["Path", "Purpose"],
        ["reports/assignment_report.md", "Written Deliverable 2 report"],
        ["reports/NYC_Taxi_Deliverable_2_Model_Optimization_Report.docx", "Word report version"],
        ["reports/metrics.json", "Dataset summary, split summary, metrics, and classification
            report"],
        ["reports/model_metrics.csv", "Baseline, logistic, and optimized model metrics"],
        ["reports/tuning_results.csv", "Validation tuning candidates and selected model"],
        ["reports/model_feature_importance.csv", "Permutation importance for optimized model"],
        ["reports/figures/", "EDA and model evaluation figures"],
        ["src/nyc_taxi_long_trip_model.py", "Main training, tuning, evaluation, and artifact
            pipeline"],
        ["notebooks/predict_new_taxi_trip.ipynb", "Notebook for new-trip prediction"],
    ]
    story.append(make_table(inventory, [2.55, 3.95], font_size=7.3))

    story.append(PageBreak())
    story.append(para("Appendix A: requirements.txt", styles["Section"]))
    story.append(code_block(requirements, styles["CodeBlockCustom"], width=96))

    story.append(PageBreak())
    story.append(para("Appendix B: Complete Main Training Pipeline", styles["Section"]))
    story.append(
        para(
            "This is the core project code that downloads/loads the data, preprocesses trips,
                tunes the model, evaluates metrics, writes artifacts, and saves the optimized
                model.",
            styles["Body"],
        )
    )
    story.append(code_block(main_code, styles["CodeBlockCustom"], width=98))

    story.append(PageBreak())
    story.append(para("Appendix C: Prediction Notebook Code Cells", styles["Section"]))
    for idx, cell_source in enumerate(prediction_cells, start=1):
        story.append(para(f"Prediction notebook code cell {idx}", styles["Subsection"]))
        story.append(code_block(cell_source, styles["CodeBlockCustom"], width=98))

    story.append(PageBreak())
    story.append(para("Appendix D: Day 2 Colab Notebook Code Cells", styles["Section"]))
    for idx, cell_source in enumerate(day2_cells[:8], start=1):
        story.append(para(f"Day 2 notebook code cell {idx}", styles["Subsection"]))
        story.append(code_block(cell_source, styles["CodeBlockCustom"], width=98))

    story.append(PageBreak())
    story.append(para("Appendix E: PDF Generation Script", styles["Section"]))
```

```
    story.append(
        para(
            "This appendix includes the script that generated this one-file submission PDF so the
                packet itself is reproducible.",
            styles["Body"],
        )
    )
    story.append(code_block(pdf_script, styles["CodeBlockCustom"], width=98))

    doc = SimpleDocTemplate(
        str(OUTPUT_PDF),
        pagesize=letter,
        rightMargin=0.72 * inch,
        leftMargin=0.72 * inch,
        topMargin=0.72 * inch,
        bottomMargin=0.72 * inch,
        title="NYC Taxi Deliverable 2 Full Submission Report",
        author="Soumith Kumar Ananthula, Varun Reddy Beesam, Vivek Doppalapudi",
    )
    doc.build(story, onFirstPage=page_footer, onLaterPages=page_footer)
    return OUTPUT_PDF


if __name__ == "__main__":
    print(build_pdf())
```